PROJECT OVERVIEW

slideotter

A local browser studio for structured decks, live review, guarded generation, materials, validation, and archived output.

The project turns deck work into a tight loop: plan, edit, attach, compare, apply, validate, archive.

Structured source

Slides, context, materials, and state stay inspectable in the project tree.

Live workbench

Preview, selector, spec editing, variants, planning, and checks share one browser loop.

Guarded output

Generated changes stay reviewable until the author applies and validates them.

TWENTY-SLIDE TOUR

Project map

The deck explains the authoring problem, storage model, live editor, material workflow, generation, planning, validation, and archive handoff.

Model

Presentations, deck context, slide specs, and materials.

Workflow

Edit live, generate candidates, compare, apply, and plan.

Handoff

Validate rendered output, refresh baselines, and archive the deck.

The browser feels direct, but the server keeps writes, rendering, and validation inside clear boundaries.

WHY THIS EXISTS

The problem is deck drift

Presentation work breaks down when source, preview, context, generated options, and final PDF live in separate loops.

Common drift

Source drift
Files stop matching what authors inspect.

Context drift
Intent and constraints disappear during edits.

Variant drift
Generated options overwrite too early.

Output drift
PDF differences appear late.

Design response

One local loop
Keep work in the browser studio.

Visible source
Keep JSON and preview close.

Gated output
Validate before archiving.

OPERATING MODEL

The studio loop is local

slideotter treats deck authoring as a local application workflow backed by explicit files and server-owned mutations.

Loop stages

Plan
Capture objective, outline, tone, and constraints.

Edit
Change text and JSON against the live slide.

Review
Compare candidates before promotion.

Ship
Build, validate, and archive.

Boundaries

Repo-local
No remote service required for local rules.

Server-owned writes
The browser requests changes.

Inspectable state
Files stay readable.

WORKSPACE MODEL

Presentations are first-class

The active deck is selected from a registry, and every presentation owns its slides, materials, context, variants, and theme state.

Storage layout

Registry
studio/state/presentations.json selects the deck.

Slides
presentations/<id>/slides.

Deck state
Context, variants, materials.

Selector cards
First-slide preview and deck facts.

User actions

Create
Start from constraints and a scaffold.

Duplicate
Copy the whole deck folder.

Delete
Remove with confirmation.

DECK INTENT

Context steers the deck

Audience, objective, tone, constraints, outline, palette, validation settings, and per-slide intent live with the active presentation.

Context files

Deck brief
presentations/<id>/state/deck-context.json.

Slide notes
Intent, must-include points, layout hints.

Theme values
Palette and font choices.

Check settings
Rule severity and media mode.

Usage rules

Nearby
Visible when needed, hidden otherwise.

Shared
Generation and validation read it.

Scoped
Each presentation owns its context.

CONTENT MODEL

Structured specs are the source

Supported slides use compact JSON documents that the studio can validate, diff, render, and update without raw source editing.

Supported families

Cover
Identity, summary, logo, cards.

TOC
Tour framing and section cards.

Content
Evidence columns and guardrails.

Summary
Checklist and resource cards.

Spec contract

Schema checked
Invalid JSON cannot save.

Readable
Specs stay compact.

Extensible
New families can be added.

LIVE EDITING

The preview edits the spec

Direct text edits and JSON edits update the active DOM preview immediately, so authors can inspect changes without a build step.

Edit path

Inline text
Double-click editable DOM fields.

JSON sheet
Valid drafts preview live.

Slide strip
Compact preview-only navigation.

Save
Persist without rebuilding.

Controls

Valid only
Save disables on parse errors.

Selected slide
Operations stay scoped.

Preview first
The slide remains dominant.

IMAGE WORKFLOW

Materials attach to slides

The studio can upload presentation-scoped images, store metadata, and attach one material to the selected structured slide.

Material path

Files
presentations/<id>/materials.

Registry
state/materials.json tracks metadata.

Slide media
Optional media object in JSON.

DOM figure
Image, alt text, caption/source.

Safety rules

Images only
PNG, JPEG, GIF, WebP.

Small uploads
Server caps file size.

Explicit attach
Upload does not alter a slide.

RUNTIME MODEL

One DOM path owns output

The browser preview, thumbnails, compare panes, PNGs, PDF export, and validation all read the same DOM-rendered slide.

Output surfaces

Studio preview
Active slide and thumbnails.

Compare panes
Current and candidate views.

PDF export
Playwright prints the DOM.

DOM validation
Rendered layout is inspected.

Runtime boundary

Single renderer
studio/client/slide-dom.ts.

Real layout
Headless browser measures output.

No split path
Avoid parallel renderers.

VARIANT LOOP

Candidates stay reviewable

Slide generation creates session-only candidates with previews, source diffs, structured summaries, and explicit apply controls.

Candidate path

Count control
Default count is five.

Modes
Ideate, wording, theme, layout.

Compare
Preview and source side by side.

Theme preview
Fonts and colors can vary.

Apply discipline

Session-only
No generated options in slides.

Media preserved
Attachments carry forward.

Explicit apply
Author controls promotion.

REVIEW WORKSPACE

Comparison is the decision surface

The compare area stays hidden until candidates exist, then focuses on current versus candidate preview, changed fields, and apply choices.

Review signals

Current
Rendered slide and JSON.

Candidate
Rendered alternative and JSON.

Change summary
Touched areas and line diffs.

Apply path
Apply with optional checks.

Decision cues

Hidden by default
No empty comparison chrome.

Direct selection
Rows choose what to inspect.

No surprise writes
Candidates apply only on command.

STRUCTURE WORKFLOW

Deck planning works above slides

Deck Planning uses the saved brief and outline to propose structural changes, shared context patches, and previewable plan candidates.

Plan outputs

Outline
Updated deck order and roles.

Deck patch
Theme, constraints, and context.

Slide actions
Insert, replace, archive, retitle.

Preview strips
Before and proposed deck.

Promotion rules

Shared settings toggle
Deck patch is optional.

Presentation paths
Diffs name active files.

Apply once
Plan changes are explicit.

CHAT LAYER

The assistant calls workflows

Workflow chat sits beside the studio, keeps selected slide text as context, and can trigger bounded operations instead of editing files directly.

Assistant inputs

Message
User intent and request.

Selection
Optional slide text context.

Slide id
Current working target.

Generation mode
Local, LLM, or auto.

Execution model

Workflow actions
Assistant triggers known operations.

Session history
Messages persist repo-locally.

Debug tucked away
Provider state is inspectable.

SAFETY MODEL

Writes stay narrow

The server allows only slide specs, presentation state, materials, repo-local studio state, and generated workflow artifacts.

Allowed targets

Slide specs
presentations/<id>/slides.

Presentation state
Context, variants, materials.

Materials
Image files beside the deck.

Generated output
studio/output artifacts.

Change discipline

Allowlist
Reject unapproved paths.

Explicit actions
Add, remove, attach, apply.

Archive separate
Publication is deliberate.

QUALITY GATE

Validation reads the rendered deck

The gate checks types, rendered layout, media behavior, slide-media preservation, deck plans, Studio layout, and render-baseline drift.

Gate coverage

Type check
Studio, server, scripts.

DOM layout
Bounds, padding, text, contrast.

Media checks
Legibility, captions, sources.

Browser workflows
Presentation and Studio smoke tests.

Failure modes

Baseline tracked
Refresh intentional visual changes.

Severity settings
Rules stay inspectable.

Fixtures
Known regressions stay covered.

ARCHIVE PATH

Output is reproducible

The same DOM runtime builds the local PDF, render pages, contact sheets, visual baselines, and checked-in archive snapshot.

Artifacts

PDF
slides/output/<presentation-id>.pdf.

Preview pages
studio/output/<id>/rendered-pages.

Baseline
studio/baseline/<id>.

Archive
archive/<presentation-id>.pdf.

Publish routine

Build first
Regenerate current output.

Gate
Run quality checks.

Archive update
Copy only after intent is clear.

ENGINEERING MODEL

The project favors maintainable code

TypeScript, focused fixtures, CI, ADRs, and explicit docs keep the studio changeable without losing the guardrails.

Maintenance surfaces

TypeScript
Studio and scripts typecheck.

Fixtures
Media, deck-plan, workflow, layout.

CI
GitHub gate mirrors local checks.

ADRs
Durable decisions stay recorded.

Working rules

Small slices
Commit between changes.

Docs aligned
Update docs with workflow changes.

No stale paths
Scan for old assumptions.

ROADMAP

Next work stays pragmatic

The near-term roadmap favors real multi-deck usage, concrete media-heavy gaps, and targeted deck-planning modes over broad editor expansion.

Next slices

Multi-deck hardening
Wait for usage before tagging.

Media validation
Deepen when real gaps appear.

Deck planning
Add modes with context patches.

Docs
Keep DOM-first guidance current.

Avoided scope

Not WYSIWYG
Do not chase full editor scope.

No second renderer
DOM remains authoritative.

No blind apply
Generation stays reviewable.

TAKEAWAY

What this enables

Deck authors get one local loop for context, structured editing, materials, review, validation, export, and archive.

- **Author with context**

  Keep objective, constraints, intent, materials, and content together.

- **Compare before apply**

  Generate candidates, inspect differences, and promote one version.

- **Ship with guardrails**

  Validate layout, media, Studio chrome, and render drift before archiving.

WHERE THE WORK LIVES

Studio app

studio/client, studio/server, scripts

Runtime path

slide JSON, materials, DOM renderer, PDF, archive